

ARQA TECHNOLOGIES

HowTo FIX Program Interfaces

January 2024



quik

Contents

1. General information 4
   1.1 FIX protocol history 4
   1.2 FIX protocol short description 4
2. Format of FIX message 5
3. Structure of FIX message 5
4. Levels of FIX protocol 6
5. Initiation of FIX session 8
6. Identification parameters of FIX session 9
7. Numbering of FIX message 9
8. Main FIX messages of Application level 10
9. Instrument identification methods 12
10. FIX program interfaces 12
    10.1 FIX adapter 13
    10.2 FIX Client Connector 14
    10.3 FIX Drop Copy 14
11. FIX program interfaces licensing 15
    11.1 FIX adapter 15
    11.2 FIX Drop Copy 17
    11.3 FIX Client Connector 18
12. FIX interface settings 19
13. FIX interface testing 22
14. FIX message examples 26
    14.1 Most relevant examples of FIX messages 31
15. FIX message configuration 33
    15.1 Error messages 35
16. Redundancy and monitoring of FIX interfaces 36
    16.1 Monitoring of connection between adapter and QUIK server 36
    16.2 Hot redundancy of QUIK server connection 37

16.3 FIX adapter / FIX Drop Copy redundancy 38
16.4 Order cancellation by number 39
17. Maintenance schedule 40
18. Interface startup keys 40
18.1 -install 40
18.2 -remove 40
18.3 -new 40
19. Diagnostics of request rejection 41
19.1 New Order Single 41
19.2 Order Cancel Request 43
19.3 Order Cancel Replace Request 44
20. Equipment and communication channel requirements 46

1. General information

FIX program interfaces enable interaction between the QUIK server and external applications / software platforms by means of the FIX protocol.

Functions:

- Connection of an external platform to the QUIK server to obtain data from the QUIK server and transmit trade instructions to QUIK.
- Data transmission from the QUIK server to the external FIX platform.

1.1 FIX protocol history

The Financial Information eXchange (**FIX**) protocol is a specialized protocol for exchanging financial information in real-time.

It was created in 1992 to enable electronic communication of equity trading data between Fidelity Investments and Salomon Brothers.

The FIX protocol is a standard in foreign trading applications that traders and brokerage systems use to receive market data and carry out trading transactions.

It is supported by the majority of largest banks, developers of brokerage systems, and the world's largest exchanges such as CME, LSE, NYSE, NASDAQ, Moscow Exchange and many others.

FIX protocol versions differ in the set of supported messages and transmitted fields. The most common versions are:

- 4.2,
- 4.4,
- 5.0,
- 5.0 SP2.

1.2 FIX protocol short description

The FIX protocol is used to send and receive trade data by using messages of a specific format.

The FIX protocol defines rules for message creation, interpretation, and the procedure of exchanging messages among trading participants.

For more information about FIX, go to www.fixtrading.org.

2. Format of FIX message

FIX field:

Each message (a FIX message) is a set of fields in the following format:

```
<tag>=<value>
```

Where:

- <tag> is a numeric identifier of a field (tag),
- <value> is a filled string.

Each field defines a trading or service entity in a FIX message. For example: … 1=L01-00000F00 … 48=LKOH … 109=ClientCode …, etc.

FIX message:

A FIX message is a set of fields delimited by the "SOH" character. Each FIX message includes the following parts:

- Header,
- Message body,
- Trailer.

3. Structure of FIX message

1. The **header** contains service parameters (the protocol version, message type, sequence number, sender, message body length). The order of header fields is the following:
   - BeginString (8) field is a protocol version. Must always be the first field in a message.
   - BodyLength (9) field is the message body length starting from the third field but not including the checksum. Must always be the second field in a message.
   - MsgType (35) field is a message type. Must always be the third field in a message.
   - Service fields that set identification parameters (for example, SenderCompID (49) and TargetCompID (56)), message sequence number (MsgSeqNum(34)) etc.
2. The **message body** contains data transmitted. For example, the following parameters are sent in FIX message body for a buy order: price, quantity, order type, instrument code, and some other related parameters for this message type.
3. The **checksum (trailer)** contains service parameters that allow checking for the message integrity and correctness. The CheckSum (10) parameter sets the value of checksum calculated by summing all characters of a message and transformed into a modulo 256. Must always be the last field in a FIX message.

Each FIX message has a specific type that is set in the tag [35] MsgType. This tag defines the purpose of a FIX message. For example, a message of the New Order Single (35=D) type is a message for registering an order. The list of message types in the FIX protocol is strictly defined and covers all trading and service operations of FIX applications. Each FIX message has strictly defined set of fields according to its type – required and optional.

Example of a FIX message for registering an order (New Order Single (35=D)) to buy 30 lots of LKOH at price 5000:

```
8=FIX.4.2|9=125|35=D|49=TEST|56=QUIK|34=295|52=20210419-
04:12:29|11=3871|21=1|22=8|48=LKOH|100=BQUOTE|40=2|44=5000|38=30|54=1|1=L01-
00000F00|10=206|
```

4. Levels of FIX protocol

The FIX protocol is defined at two levels of trade interaction, session and application.

The **Session level** protocol ensures identification of parts of the interaction, guaranteed delivery and sequential processing of messages, connection status control, and session recovery in case of failures. This level includes the following message types:

Message	Message type
Heart Beat	0
Logon	A
Logout	5
Test Request	1
Resend Request	2
Reject	3
Sequence Reset	4

Description of session-level message types:

- **Logon** (A) initiates a session and a connection. Transmitted both directions.
- **Heartbeat** (0) monitors the status of a connection. Transmitted both directions.
- **Test Request** (1) monitors the status of a connection. Transmitted both directions. The other endpoint should reply with a marked Heartbeat.
- **Sequence Reset** (4) is used to reset the message sequence (Reset mode) and during retransmission for missed administrative messages (Gap Fill mode). Transmitted both directions.

- **Resend Request** (2) indicates that a specific message sequence has to be retransmitted. Transmitted both directions. Supported but is not a necessary part of the protocol.
- **Reject** (9) indicates if there is an incorrectly transmitted or unknown message from the other side. Transmitted both directions.
- **Logout** (5) initiates and approves disconnection. Transmitted both directions.

The protocol of trade communication (Application level) allows sending FIX messages of various types: order registration, order/trade reports reception, market data reception etc. This level includes the following message types:

Message	Message type
New Order Single	D
Order Cancel Request	F
Execution Report	8
Trade Capture Report	AE
Trade Capture Report Ask	AR
Trade Capture Report Request	AD
Order Cancel Reject	9
Order Cancel Replace Request	G
Order Mass Cancel Request	q
Market Data Request	V
Market Data Request Reject	Y
Market Data Incremental Refresh	X
Market Data – Snap Shot / Full Refresh	W
Security Definition Request	c
Security Definition	d
Order Status Request	H
Order Mass Status Request	AF
Security Status Request	e
Security Status	f
User Request	BE
User Response	BF
Request For Positions	AN

Message	Message type
Request For Positions Ack	AO
Position Report	AP
Quote Request	R
Quote Request Reject	AG
Quote	S
Quote Cancel	Z
Quote Status Report	AI
Indication of Interest	6
QuoteAcknowledgment	b

Description of some types of application-level messages:

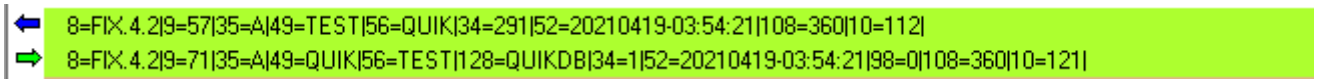
- **New Order Single** (D) is sent by a client to the trading system to register an order.
- **Order Cancel Request** (F) is sent by a client to the trading system to request cancellation of an active order.
- **Order Cancel Replace Request** (G) is sent by a client to the trading system to request replacement of an active order considering the executed quantity.
- **Order Cancel Reject** (9) is transmitted from the trading system to a client and contains a reason for not cancelling an order.
- **Execution Report** (8) is transmitted from the trading system to a client and contains information about order state change.
- **Market Data Request** (V) is sent by a client to the trading system to receive a snapshot of market data.
- **Market Data Request Reject** (Y) is transmitted from the trading system to a client and contains a reason for rejection to provide market data.
- **Market Data Snap Shot / Full Refresh** (W) is transmitted from the trading system to a client and contains market data.
- **Security Definition Request** (c) is sent by a client to the trading system to receive the dictionary of instruments.
- **Security Definition** (d) is transmitted from the trading system to a client and contains the dictionary of instruments.

5. Initiation of FIX session

Before exchanging FIX messages, a FIX client and a FIX interface initiate a FIX session. The session is initiated from two sides:

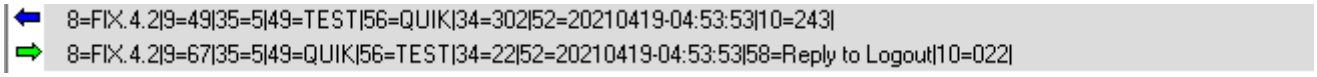
- **Initiator** establishes a network connection and initiates a session by sending a **Logon** (35=A) FIX message.
- **Acceptor** accepts a connection from the other side and, if information in the received **Logon** message is correct, sends a response message **Logon**. Otherwise, a connection breaks down.

After exchanging **Logon** FIX messages, a FIX session is considered established, a FIX client and a FIX interface can exchange messages of other types.



To check the session state, a FIX client and a FIX interface repeatedly exchange HeartBeat (35=0) messages after a certain period of time. If there is no response to a Heartbeat, a FIX session breaks down.

To initiate the session closure, a participant sends a Logout (35=5) message and after receiving a response Logout breaks a connection.



6. Identification parameters of FIX session

The parameters of FIX session identification are set in tags [49] SenderCompID and [56] TargetCompID. Values of these tags are set in the main configuration file of FIX interface in a section of [fix_connect_<ConnectionID>] view.

```
[fix_connect_*]
…
SenderCompID=QUIK
TargetCompID=USER
…
```

For two participants (FIX client and FIX interface) the parameters are set as follows (mirrored):

```
QUIK.SenderCompID = USER.TargetCompID
USER.SenderCompID = QUIK.TargetCompID
```

7. Numbering of FIX message

All incoming and outgoing FIX messages within a session are numbered separately. Each subsequent message should have a number one greater than the previous one.

Incoming and outgoing sequence numbers are controlled by the FIX interface and the FIX client.

If the number of a received message is greater than expected, a participant has to send the Resend Request (35=2) message with the missing range of messages set in tags [7] BeginSeqNo and [16] EndSeqNo to get missing messages.

If the number of a received message is fewer than expected, the FIX interface closes a session.

To reset the numbering within the current session, a FIX client has to send the Sequence Reset (35=4) message.

8. Main FIX messages of Application level

- **New Order Single (35=D)**
  Request to register an order. The ClOrdID (11) field contains a unique string identifier of the request. As a response to the New Order Single, the Execution Report (with the ClOrdID (11) from the original request) that contains a response to the request is sent.
- **Order Cancel Request (35=F)**
  Request to cancel an order. The ClOrdID (11) field contains a unique identifier.
  The OrigClOrdID (41) field contains the identifier of an order to be cancelled. In some trading modes, if after receiving a cancellation request for an order it was found and cancelled the trading system sends two Execution Report message (with identifier of order cancellation request). The first one has the status (OrdStatus(39) field) Pending_Cancel (6) that indicates that cancellation is being prepared; the second one has the status Cancelled (4) that indicates that an order is cancelled. If any errors occur, the Order Cancel Reject (with identifier of order cancellation request) message is sent.
  In a solicited cancel, the tags [11] and [41] are transmitted in the Execution Report according to the request: the original identifier of an order is transmitted in the tag [41], the request identifier is in the tag [11].
- **Order Cancel Replace Request (35=G)**
  Request to replace an order considering the executed quantity. The ClOrdID (11) field contains a unique identifier of a new order. The OrigClOrdID (41) field contains the identifier of an order to be replaced.
  If an order being replaced was partially executed, then a new order will be submitted at a volume less than the specified by the executed volume. This request can be used to change the order price. Side, instrument and other parameters cannot be changed.
- **Order Cancel Reject (35=9)**
  This message is received if there is an error while cancelling an order. The ClOrdID (11) field contains an identifier of an unsuccessful request.
- **Execution Report (35=8)**
  The report on order state change and trade reception is transmitted in a single Execution Report. The ClOrdID (11) field contains an identifier of order state change request. The main fields are:

    - OrderQty (38) is total quantity in an order;
    - CumQty (14) is executed quantity in an order;

- LastShares (32) is quantity of current execution;
- LeavesQty (151) is balance of an order. Calculated as difference between total quantity (tag [38]) and executed quantity (tag [14]).

- **Market Data Request (35=V)**
  Request to subscribe to / refuse subscription to market data transmission (bid, offer, last trade etc). The MDReqID (262) field contains a unique string identifier of the request. The SubscriptionRequestType (263) field contains the subscription type:
  - Snapshot (0);
  - Snapshot + Updates (1);
  - Disable previous Snapshot + Update Request (2) – Unsubscribe.

  The MarketDepth (264) field contains depth of the transmitted Level II Quotes:
  - Full Book (0);
  - Top of Book (1);
  - Report best N price tiers of data (N > 1).

  The NoMDEntryTypes (267) field contains the number of requested parameters. Codes of required parameters are specified in the MDEntryType (269) fields.

Name	Tag	Comment
→ MDEntryType **	269	Valid values: _ 0 – bid; _ 1 – offer; _ 2 – last, last trade price; _ 3 – Index Value; _ 4 – open, openin price; _ 5 – close, closing price; _ 6 – settleprice, previous settlement price; _ 7 – high, trade maximum price; _ 8 – low, trade minimum price; _ 9 – waprice, previous valuation; _ r – crossrate; _ s – real minstep, minimum price step; _ Q – Auction Clearing Price; _ T – theorprice, theoretical price; _ x – buydepo, seller's collateral; _ y – selldepo, buyer's collateral; _ O – Official Current Price

- **Market Data Request Reject (35=Y)**
  This message is received if the system receives an incorrect Market Data Request (for example, if a subscription with such identifier already exists). The MDReqID (262) field contains the problematic request identifier.

- **Market Data – Snap Shot / Full Refresh (35=W)** and **Market Data Incremental Refresh (35=X)**
  Market data is transmitted in this message.
- **Security Definition Request (35=c)**
  Request of instruments dictionary. The SecurityReqID (320) field contains the unique request identifier.
- **Security Definition (35=d)**
  This message contains the dictionary (or its part) of instruments. The TotalNumSecurities (393) field contains the number of instruments within a received message.

9. Instrument identification methods

The instrument identification method is set in the tag [22] IDSource in a FIX message. The tag has the following possible values:

- 1 – by CUSIP code of an instrument;
- 2 – by SEDOL code of an instrument;
- 4 – by ISIN code of an instrument;
- 5 – by RIC code of an instrument;
- 6 – by ISO code of an instrument;
- 8 – by code of an instrument;
- 100 – by short name of an instrument.

The instrument identifier is set in the tag [48] SecurityID. Short name of an instrument is set in the tag [55] Symbol.

If the list of instruments loaded by the Interface there is more than one instrument with the same identifier name, the Interface rejects a message with diagnostics: "Ambiguously security specification". In this case specify the code of a class or market in the tag [100] ExDestination to identify an instrument.

10. FIX program interfaces

FIX program interfaces enable interaction between the QUIK server and external applications / software platforms by means of the FIX protocol.

The list of QUIK program interfaces:

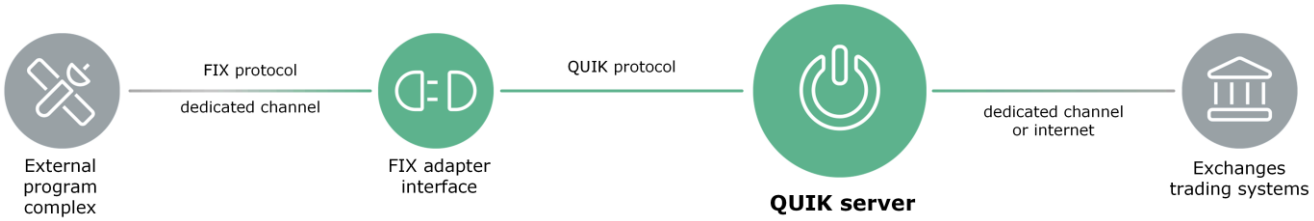
- FIX adapter program interface;
- FIX Client Connector program interface;
- FIX Drop Copy program interface.

Functions:

- Connection of an external platform to the QUIK server to obtain data from the QUIK server and transmit trade instructions to QUIK (FIX adapter and FIX Client Connector).
- Data transmission from the QUIK server to the external platform (FIX Drop Copy).

10.1 FIX adapter

The FIX adapter program interface serves for connecting external applications to the QUIK server in order to obtain data and transmit trade instructions. In technological terms, the interface is a FIX API to the QUIK server.



The interface allows getting from the QUIK server market data, reference data, anonymous trades data as well as data on orders and trades made via this particular interface.

This interface also allows you to obtain information about clients' positions on the QUIK server. Please note that if you need to translate a large number of client positions to an external FIX system (for example, more than 10,000 lines), then to solve this problem you must use a separate Exchange Information Export Module (QuikExport). More details on this Module - https://arqatech.com/ru/products/quik/modules/integration-solutions/market-data-export/

The interface provides access to any trading system, trading mode or order type supported by a respective QUIK server as well as to the full QUIK software complex's functionality of online pre-trade operation control.

The interface supports operations with algo orders of the Algorithmic trading module and OMS orders of QUIK OMS.

The interface does not support REPO operations (except anonymous REPO with CCP). NDM operations are supported.

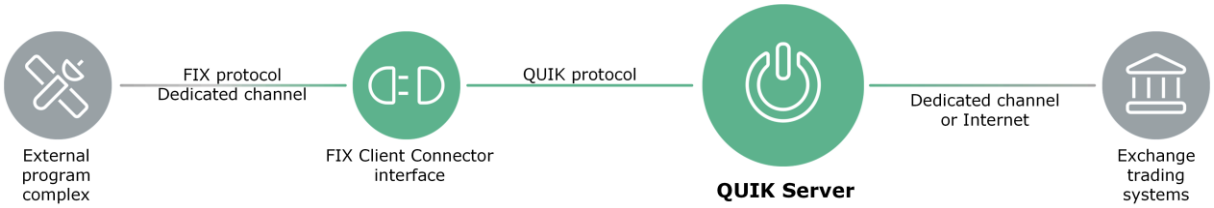
Two FIX connections can be created simultaneously: main and additional (which is defined by the **IsPrimary** setting) connections. One of them is used to send transactions, another is for subscribing to market data. As these connections operate independently, market data subscription does not affect transaction sending.

> **We recommend that you use two separate FIX sessions if one FIX adapter interface is used to receive market data and send transactions.**

This interface supports modification of FIX message in the .xml scheme (for details, see section 15 "FIX message setting").

10.2 FIX Client Connector

The FIX Client Connector program interface serves for connecting external client applications in order to obtain data and transmit trade instructions to the QUIK server. In technological terms, the interface is a FIX API to the QUIK server.



The interface allows getting from the QUIK server market data, reference data, anonymous trades data as well as data on orders and trades made via this particular interface. Also, the interface allows getting information on client positions on the QUIK server.

The interface provides access to any trading system, trading mode or order type supported by a respective QUIK server as well as to the full QUIK software complex's functionality of online pre-trade operation control.

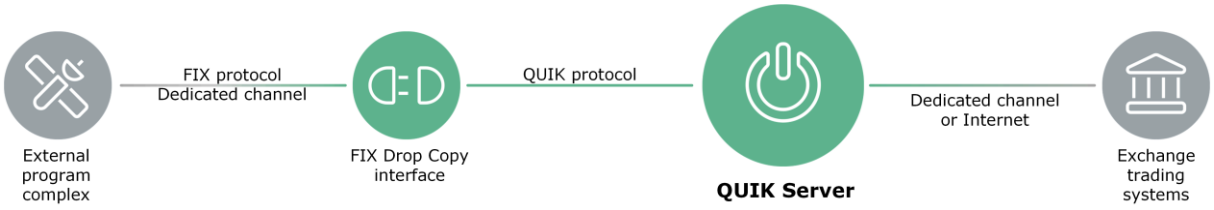
The interface does not support NDM, REPO operations (except anonymous REPO with CCP), algo and OMS orders.

The program interface allows servicing up to 10 client accounts on spot and derivatives markets.

The program interface allows establishing a single FIX session. This interface does not support modification of FIX message in the .xml scheme.

10.3 FIX Drop Copy

The interface allows getting from the QUIK server market data, reference data as well as data on orders and trades for a particular account or a group of accounts to an external system (middle-, back-office or other systems used by the broker) via FIX protocol. In technological terms, the interface is a FIX API to the QUIK server.



This interface does not support sending transactions to the QUIK server.

Also, the interface allows getting information on client positions on the QUIK server.

Please note that if you need to translate a large number of client positions to an external FIX system (for example, more than 10,000 lines), then to solve this problem you must use a

separate Market Data Export module (QuikExport). More details on this Module - https://arqatech.com/ru/products/quik/modules/integration-solutions/market-data-export/

FixDropCopy must be used to translate the quote table to external FIX systems.

> **One instance of the Interface supports only one FIX session.**

11.FIX program interfaces licensing

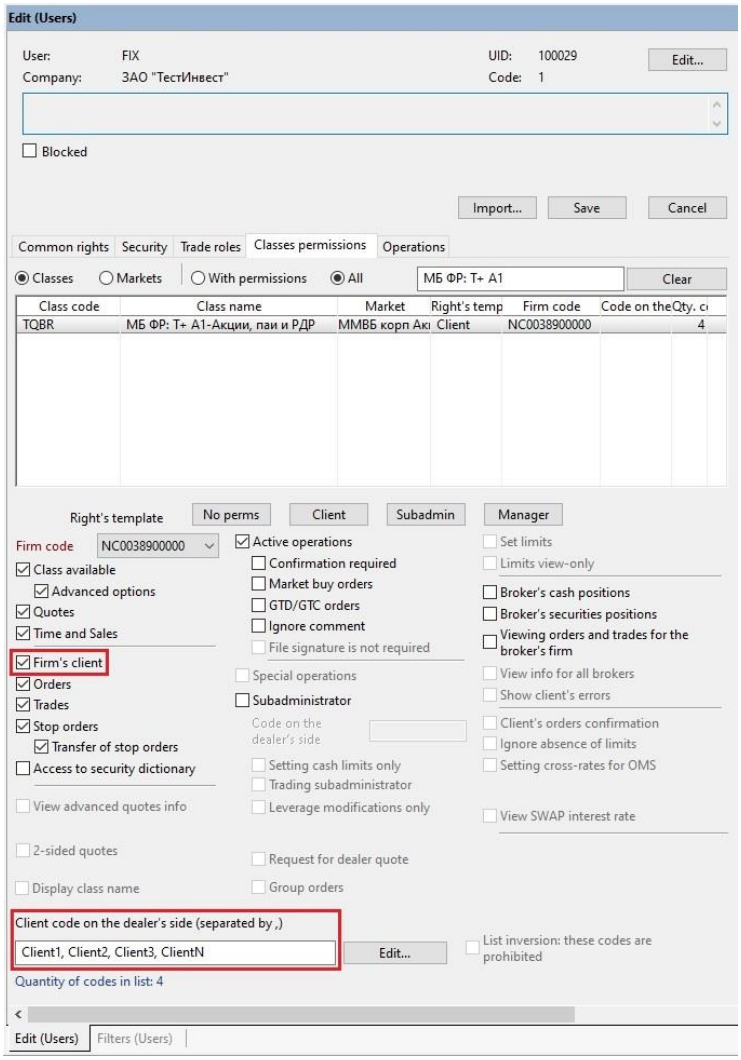
This section covers licensing options of FIX program interfaces.

11.1 FIX adapter

There are three types of FIX adapter program interface (FIX adapter BE) with a limit on:

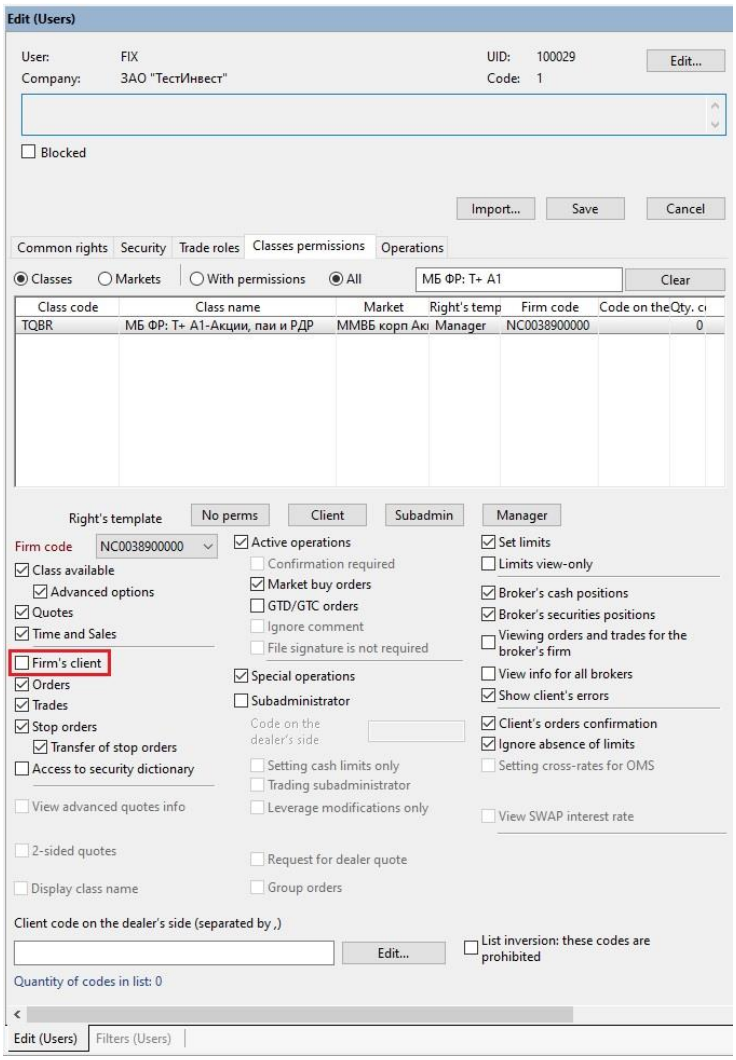
- number of serviced client accounts (basically, up to 50),
- number of processed transactions (basically, up to 10 000 per month) – this type of the interface is used if the QUIK server is outsourced from the ARQA Technologies' data center,
- number of processed transactions (basically, up to 500 per day) – this type of the interface is used if the QUIK server is installed at broker's capacities.

For FIX adapter with licensing by the **number of client accounts**, in the QUIK Administrator program assign the **Client** template and list client codes in quantity that does not exceed the number of licenses.



The control of licenses is carried out by the QUIK server when the FIX adapter interface connects to it. In case of violation of the licensing policy, the QUIK server forcibly disconnects FIX adapter and writes out the diagnostics to the main QUIK server log file.

For FIX adapter with licensing by the **number of transactions** (FIX adapter BE), in the QUIK Administrator program assign the **Manager** template. Thus, a service user will be allowed to conduct operations by all client accounts on the QUIK server.

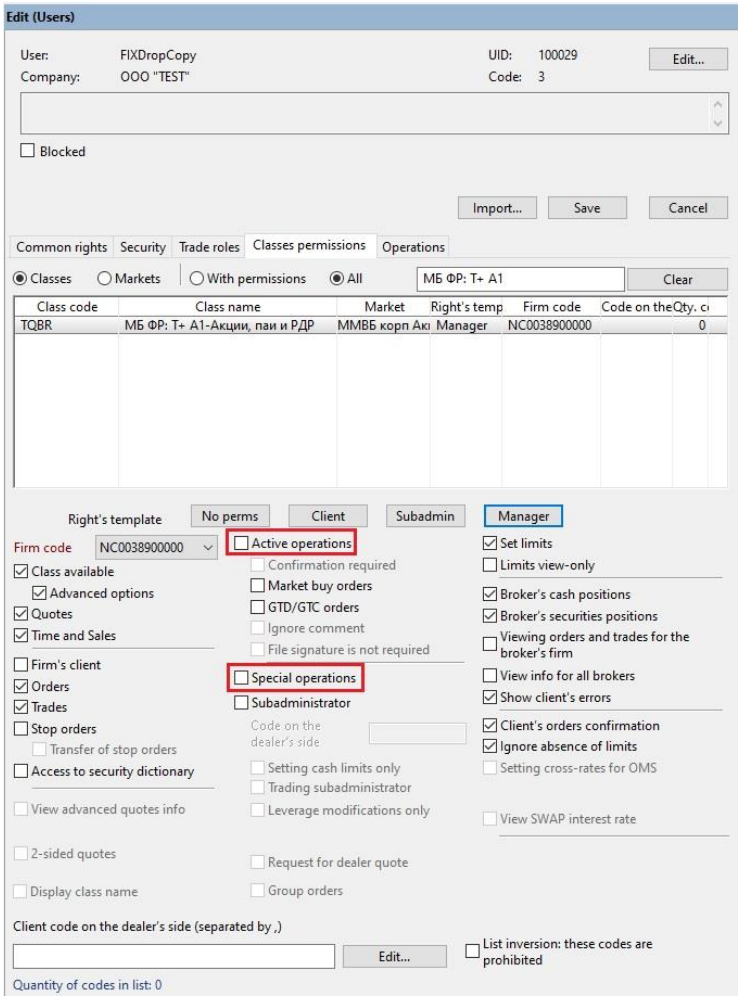


The control of licenses is carried out by the QUIK server (it counts the number of transactions). In case of violation of the licensing policy, the QUIK server forcibly disconnects FIX adapter BE and writes out the diagnostics to the main QUIK server log file.

> **When counting the number of transactions, the QUIK server considers any transaction sent via the FIX adapter BE: for order registration, cancellation, replacement (including rejected by the QUIK server).**

11.2 FIX Drop Copy

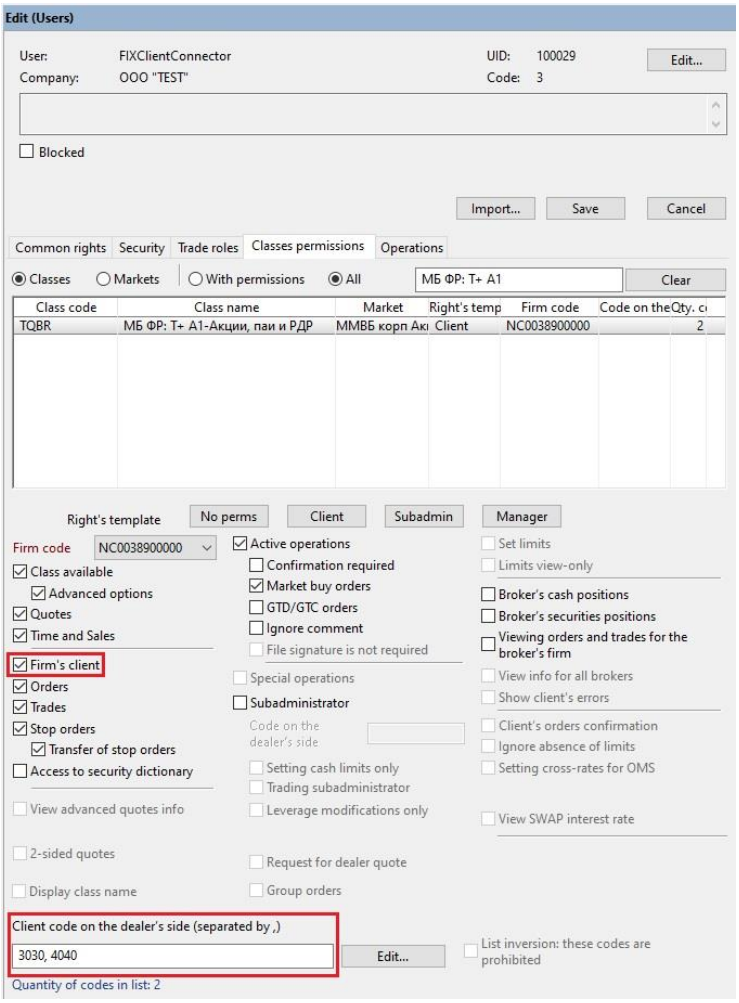
This interface does not support sending transactions to the QUIK server. In the QUIK Administrator program clear the **Active operations** and **Special operations** check boxes by **all** trading modes for a service user of FIX Drop Copy.



The control of licenses (the check of permissions on active operations) is carried out by the QUIK server when the FIX Drop Copy program interface connects to it. In case of violation of the licensing policy, the QUIK server forcibly disconnects FIX Drop Copy and writes out the diagnostics to the main QUIK server log file.

11.3 FIX Client Connector

For FIX Client Connector, in the QUIK Administrator program assign the **Client** template and list client codes (not more than 10).



The control of licenses is carried out by the QUIK server when the FIX Client Connector program interface connects to it. In case of violation of the licensing policy, the QUIK server forcibly disconnects FIX Client Connector and writes out the diagnostics to the main QUIK server log file.

12.FIX interface settings

The main settings of FIX interfaces are set in the main configuration file with an .ini extension. This sub-section covers examples of interface main settings that can be used to connect to the QUIK server.

- [Files] section:

```
; storage for FIX session parameters (counters)
SystemInfo=Data\SystemInfo.xml
; main log file
LogFileName=Import\quikfix.log
; log file rotation depth
LogDepth=3
; orders file
OrderStorage=Import\orders.data
```

```
; trades file
TradeStorage=Import\trades.data
; file of trades for execution
UsTradeStorage=Import\ustrades.data
; orders history file (information about active GTC and GTD orders)
OrderHistory=Import\OrderHistory.data
; the file contains correspondences of instruments and their alternative codes
SecurityInfo=SecInfo.xml
; the file contains correspondences of instrument types and values transmitted
in the tag [167] SecurityType
SecTypes=SecTypes.xml
; severity level mask of debugging messages written to a log (HIGH, NORMAL, LOW)
DebugMsgUrgency=HIGH;
```

- [General] section:

```
; indicates that date and time in orders and trades are to be converted to GMT
SendTradeTimeGMT=1
; session change tine in format <HHMMSS>
ClearingTime=030000
; determines how the comment field should be filled in an order being submitted
CommentType=3
```

- [fix_connect_*] section:

```
; connection type: IN – incoming; OUT (default) – outgoing;
Type=IN
; connection port
Port=15100
; identifier of a company that sends a message
SenderCompID=QUIK
; identifier of a company that receives a message
TargetCompID=TEST
; Interface messages storage
Storage=Import\storage.data
; FIX message scheme used
Schema=Rules\Default30.xml
; FIX protocol version
ApplVerID=FIX42
; interval for sending heartbeat in seconds
Keepalive=30
; indicates whether this connection is main or additional
IsPrimary=1
```

- [Markets] section is used to set correspondences between markets and instrument classes:

```
MICEX=|EQBREMU|QME_CFD|
RTS=|GTS|SPBFUT|
MICEX1=|TQBR|
OMS=|OMS|OMS_BOND|
SICETW=|BQUOTE|BPSEQ|
POS=|POS_EQ|POS_FI|
```

- [Quik] section contains the list of instrument classes in which a FIX client can request market data and submit / cancel / replace orders:

```
Classes=|TQBR|SPBFUT|EQBS|BQUOTE|TQBR|OMS|QME_CFD|OMS_BOND|POS_EQ|POS_FI|
```

- [QuikApi] section is used to identify a QUIK server connection:

```
; login
User=FIX
; password
Password=123456
; name of the file that contains settings to connect to QUIK server
Ini=QuikApi.ini
; path to the directory of QuikApi library log files
LogDir=QuikAPI
; path to the directory of QuikApi library data storage files
DatDir=QuikAPI
; system language identifier. Valid values: 9 (default) – English; 25 – Russian
Language_ID=25
```

Settings of connection of the service user to the QUIK server are set in the QuikApi.ini file.

- [connect] section:

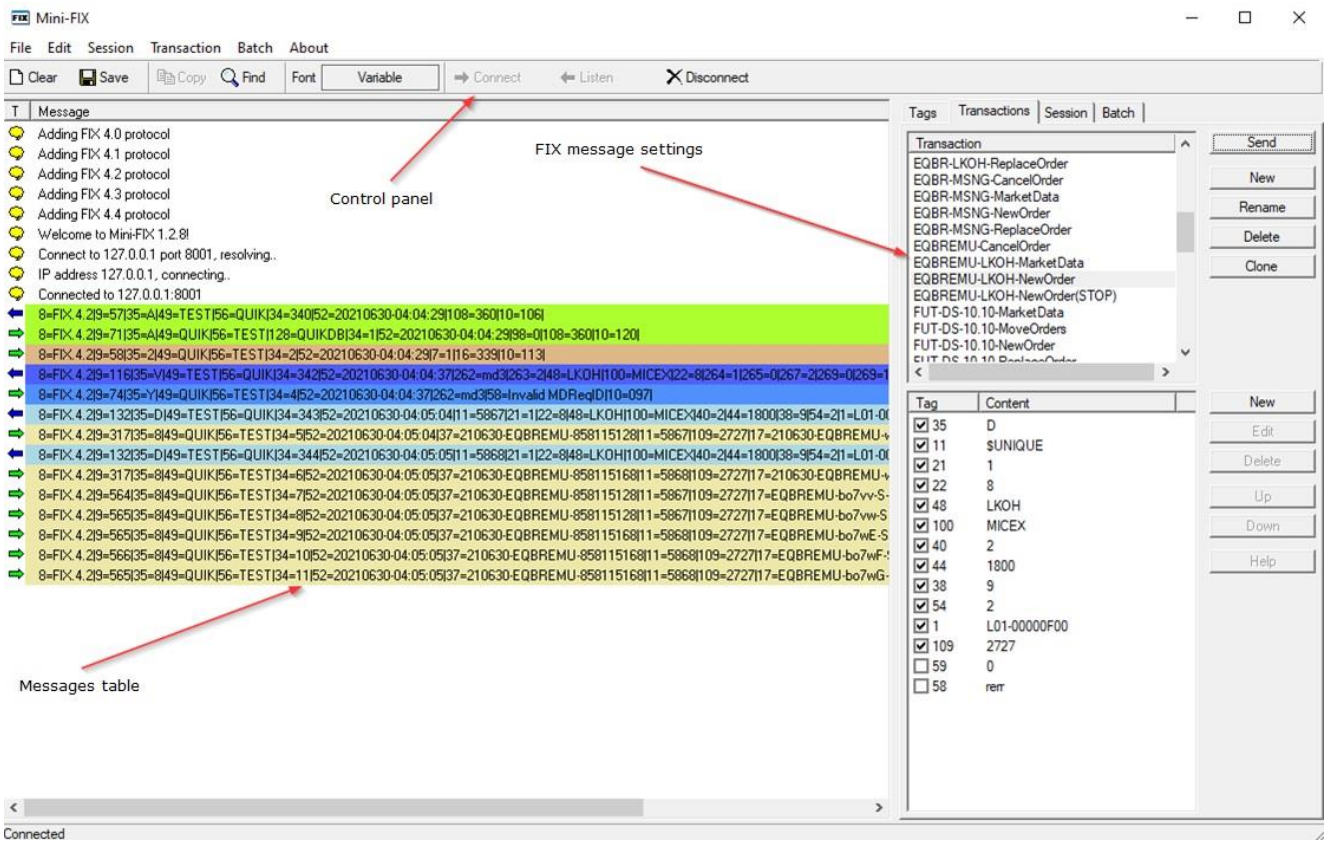
```
; IP address of QUIK server
trade-server-ip-address=127.0.0.1
; port of QUIK server
port=15100
; key of QUIK server
auth-name=quikserver
; time interval (in seconds) after which QUIK server disconnects from the interface
in case of any connectivity problems
check-tmo=150
```

- [transport] section:

```
module=wt_dtcp.dll
```

13.FIX interface testing

Mini-FIX, a GUI application, can be used to test FIX applications. This software is free and publicly available. The program allows sending and receiving FIX messages.



Program interface:

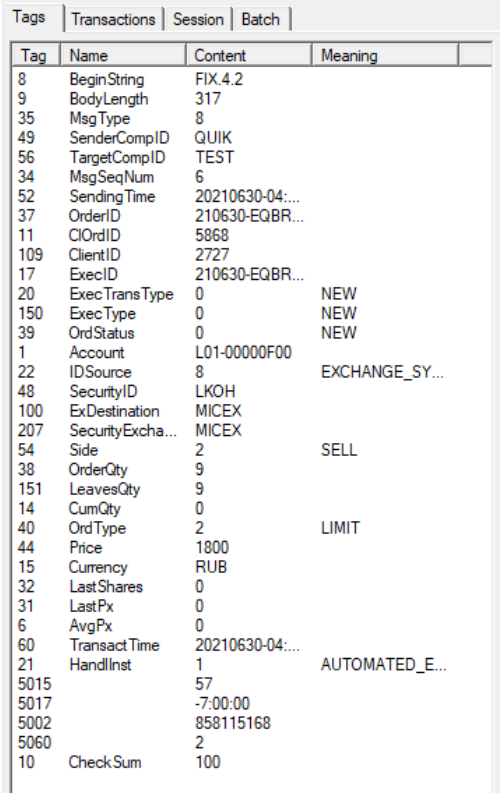
- Messages table is the program log to which service, incoming and outgoing messages are written and highlighted in color depending on FIX message type.
- Control panel in which there are buttons to manage logging and connection/disconnection.
- Tabs with tables and settings are used to set up FIX messages.

Tags tab

The Tags tab shows the table of FIX message details. It is used for a more convenient visual representation of FIX messages. Consists of 4 columns:

- **Tag** – field tag,
- **Name** – field name,
- **Content** – field value,
- **Meaning** – description.

To display a FIX message in this table, select it in the Messages table.

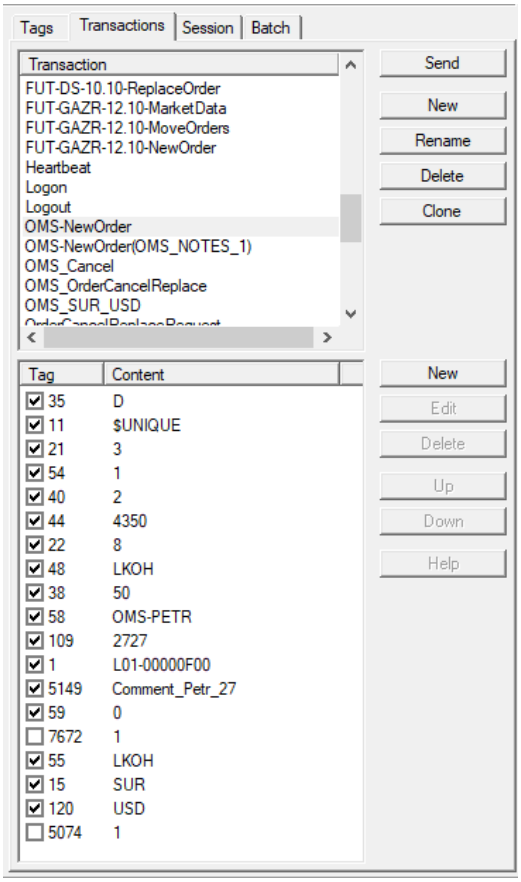


Tags | Transactions | Session | Batch

Tag	Name	Content	Meaning
8	BeginString	FIX.4.2	
9	BodyLength	317	
35	MsgType	8	
49	SenderCompID	QUIK	
56	TargetCompID	TEST	
34	MsgSeqNum	6	
52	SendingTime	20210630-04:...	
37	OrderID	210630-EQBR...	
11	ClOrdID	5868	
109	ClientID	2727	
17	ExecID	210630-EQBR...	
20	ExecTransType	0	NEW
150	ExecType	0	NEW
39	OrdStatus	0	NEW
1	Account	L01-00000F00	
22	IDSource	8	EXCHANGE_SY...
48	SecurityID	LKOH	
100	ExDestination	MICEX	
207	SecurityExcha...	MICEX	
54	Side	2	SELL
38	OrderQty	9	
151	LeavesQty	9	
14	CumQty	0	
40	OrdType	2	LIMIT
44	Price	1800	
15	Currency	RUB	
32	LastShares	0	
31	LastPx	0	
6	AvgPx	0	
60	TransactTime	20210630-04:...	
21	HandlInst	1	AUTOMATED_E...
5015		57	
5017		-7:00:00	
5002		858115168	
5060		2	
10	CheckSum	100	

Transactions tab

The tab is used to create, edit and send the user's FIX messages. It contains two tables: the Messages table above and the Fields table below which shows the list of fields of a message selected in the table above. Next to the tables there are buttons for managing FIX messages.



Buttons on the right of the Messages table are used to manage messages:

- **Send** – send a selected message,
- **New**/**Rename**/**Delete** – create/rename/delete a message,
- **Clone** – create a message copy.

Buttons on the right of the Fields table are used to edit the selected message:

- **New**/**Edit**/**Delete** – add/edit/delete a field,
- **Up**/**Down** – swap the selected field and the previous/next one,
- **Help** – display help for the selected field to know the field name, data type and available values.

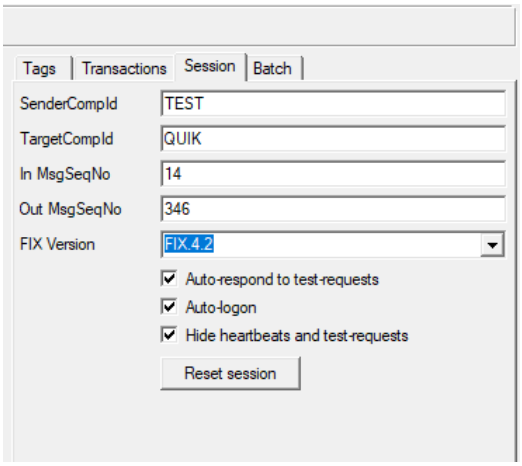
> **The Mini-FIX program automatically fills in the service fields so there is no need to add the following fields: BeginString (8), BodyLength (9), SenderCompID (49) TargetCompID (56), MsgSeqNum (34), SendingTime (52), CheckSum (10).**

<Preset words> can be used as field values. When sending a message, they will be substituted with the set values:

- $UNIQUE – a unique value,
- $LASTUNIQUE – previous unique value,
- $TIMESTAMP – GMT in FIX format.

Session tab

The tab contains session parameters settings: identification parameters, message numbers, and the protocol version.



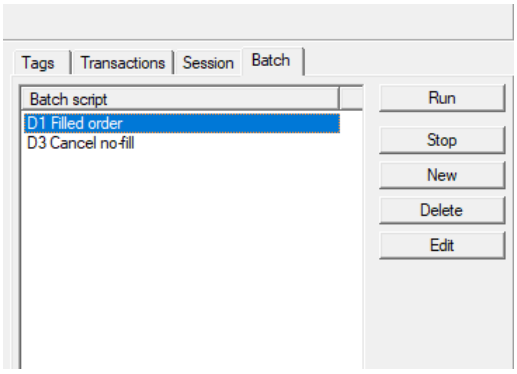
Use check boxes to enable/disable the following settings:

- **Auto-respond to test-request** – respond automatically to TestRequest,
- **Auto-logon** – send Logon automatically after establishing a connection,
- **Hide heartbeats and test-requests** – hide HeartBeat and TestRequest messages.

The **Reset Session** button resets FIX message numbering.

Batch tab

The tab contains instruments to manage scripts for automated testing. The script allows sending specific messages and analyze the result.



For the program to send a message, type in the string:

```
Send:<message name in the Messages table>
```

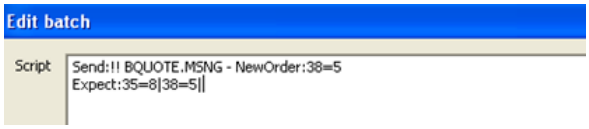
To use other field values, change a record as follows:

```
Send:<message name in the Messages table>: <tag1>=<new value1>|<tag2>=<new value2>|…|<tagN>=<new valueN>
```

To set the expected result, add a record of the following view:

```
Expect:<tag1>=<expected value1>|<tag1>=<expected value1>|…|<tagN>=<expected valueN>
```

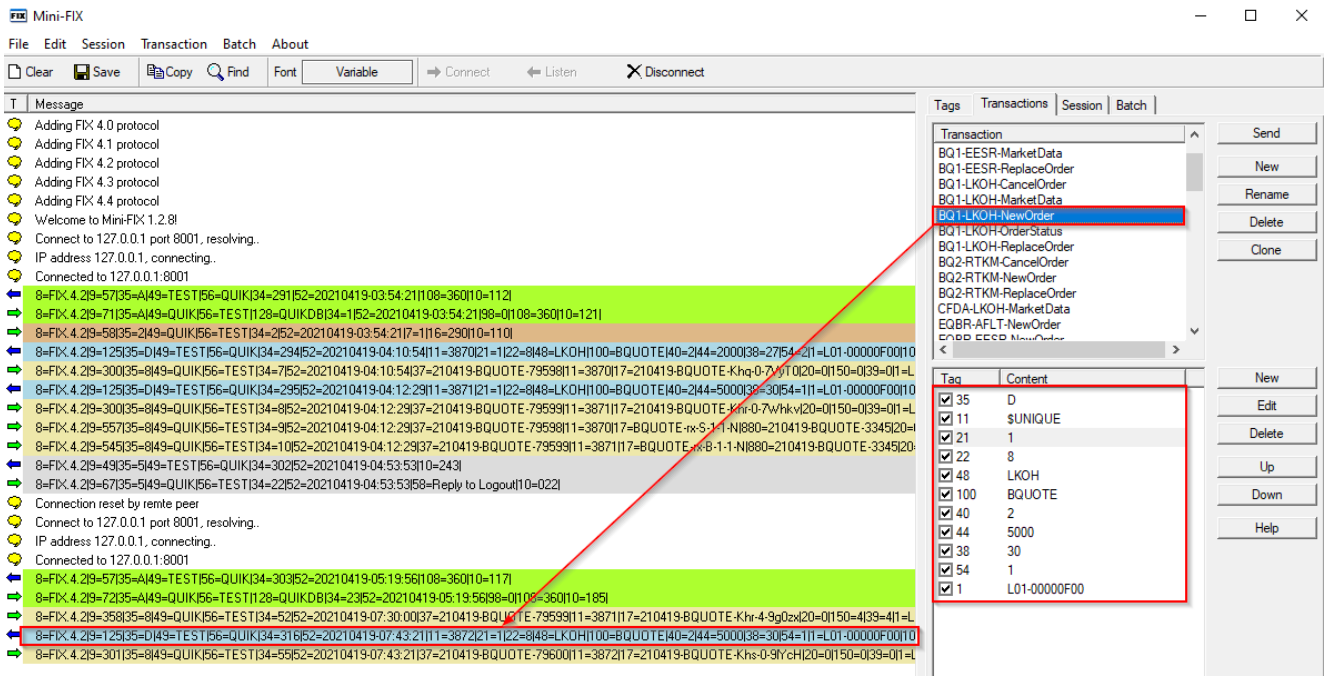
Script example:



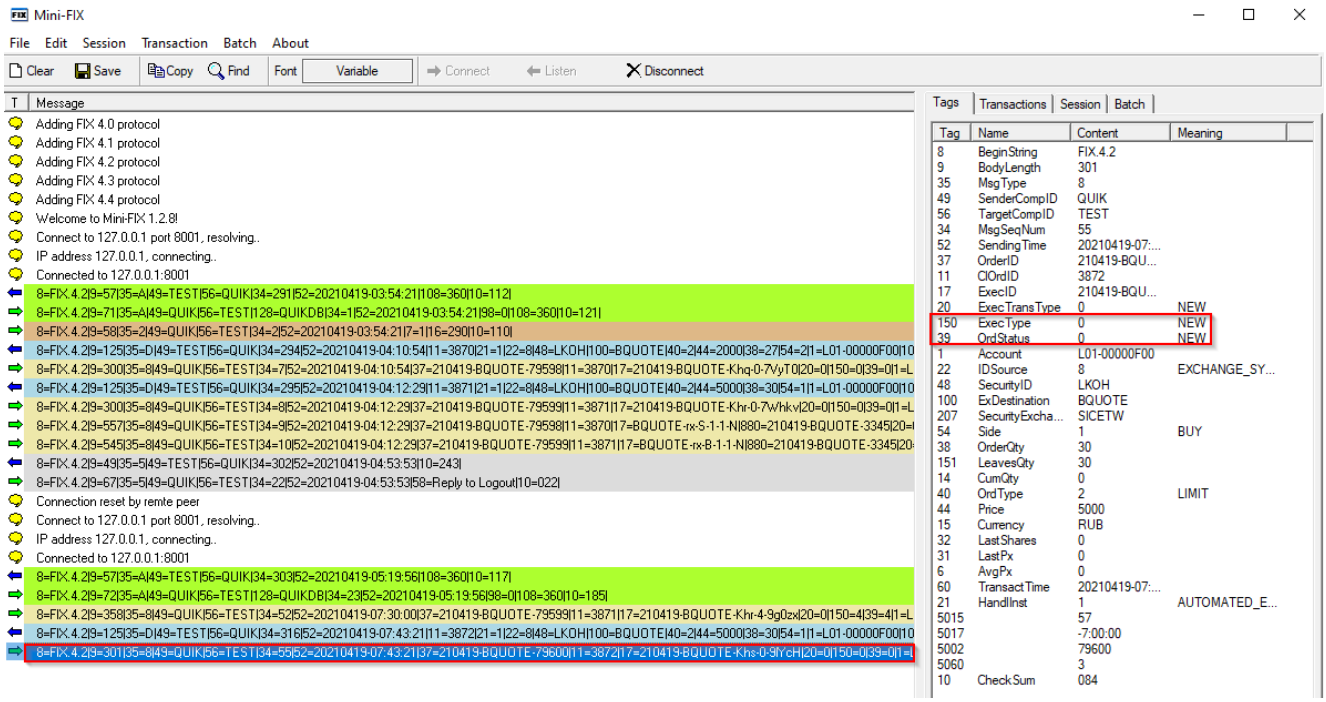
This script sends the message "!! BQUOTE.MSNG – NewOrder" with the value 5 in the field 38. ExecutionReport with the value 5 in the field 38 is expected as a response.

14.FIX message examples

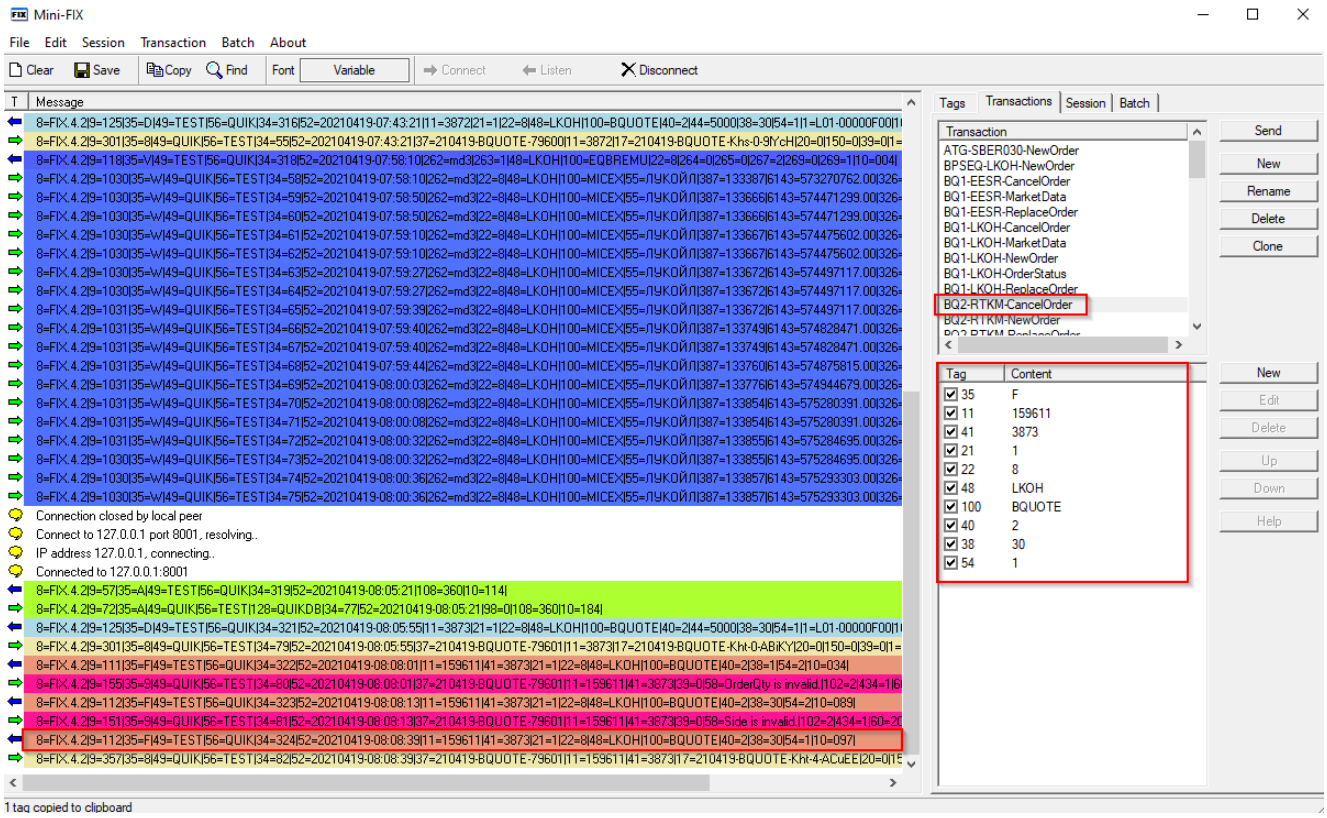
- **New Order Single (35=D)** – order registration.



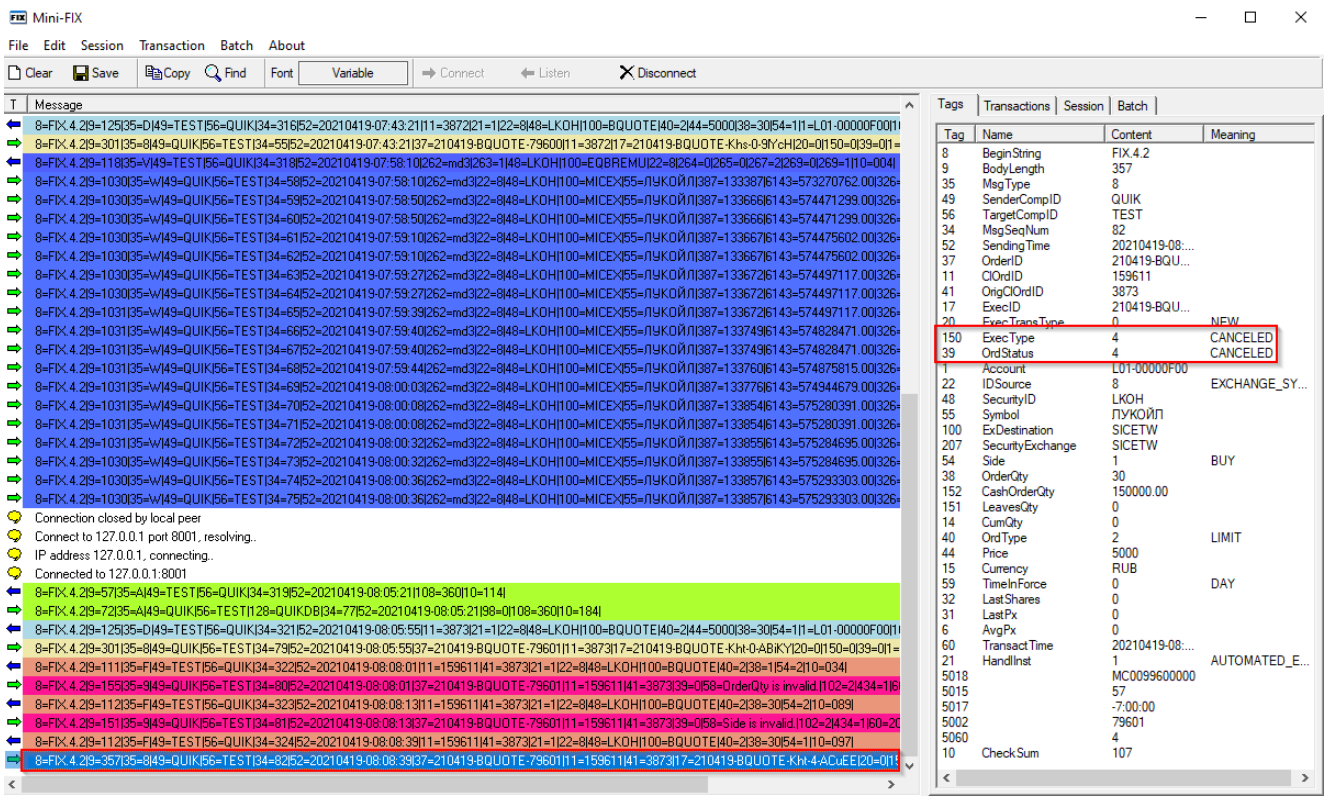
- **Execution Report (35=8)**. Order/trade status is transmitted in tags [39] and [150].



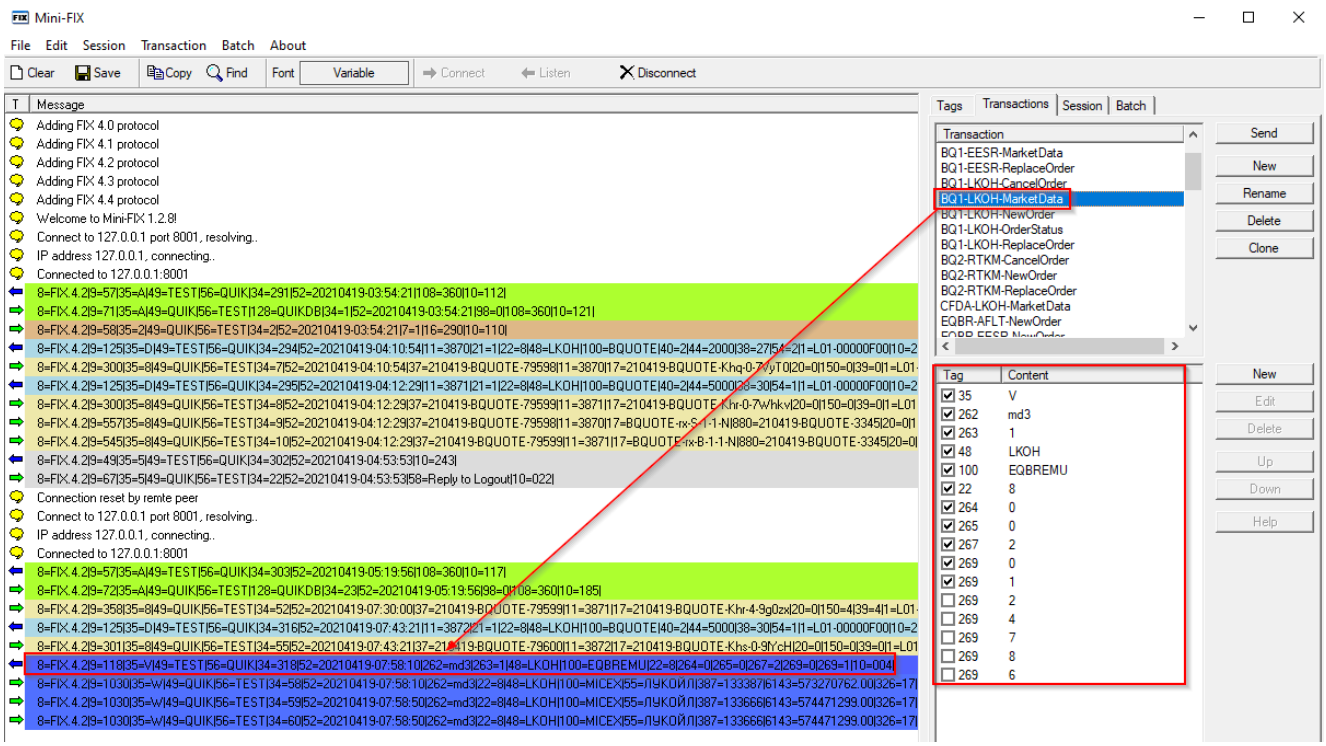
- **Order Cancel Request (35=F)**. The request to cancel an order. The message should contain a unique identifier ClOrdID (11). An identifier of an order to be cancelled is specified in OrigClOrdID (41).



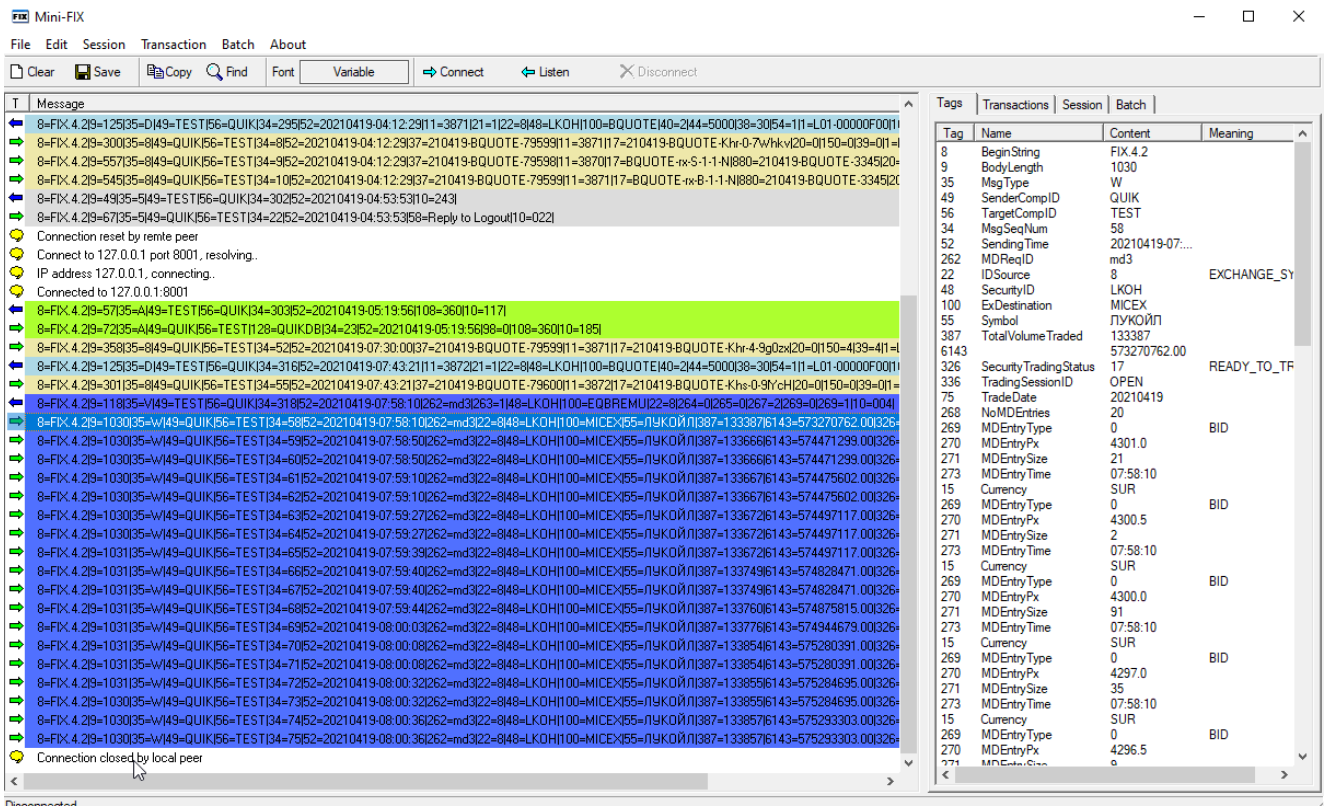
Execution Report (35=8) is transmitted as a response. The status is transmitted in tags [39] and [150].



- **Market Data Request (35=V)**. The request to subscribe/refuse to subscribe on market data transmission (bid, offer, last trade etc). A unique string identifier of the request is specified in the MDReqID(262) field.

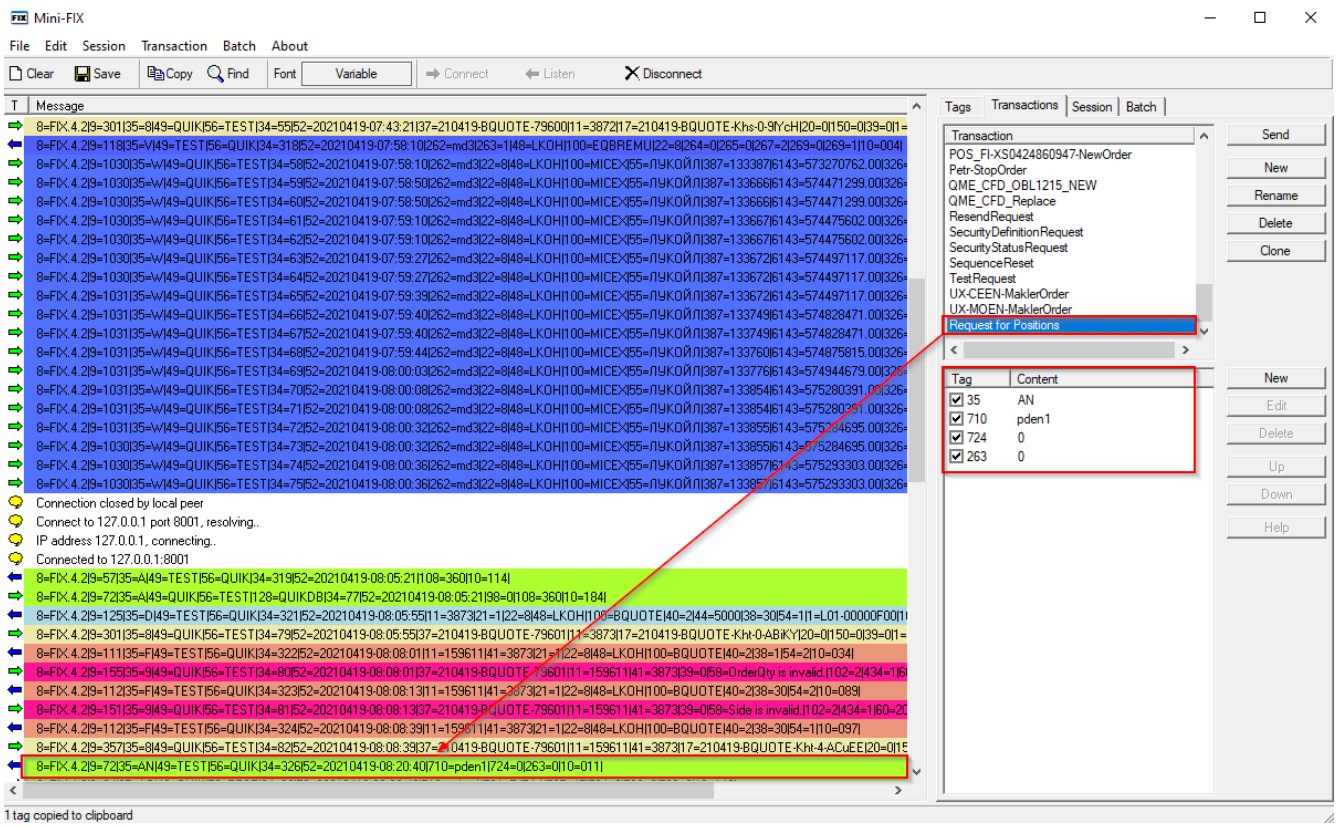


The **Market Data – Snap Shot / Full Refresh (35=W)** message is transmitted as a response.

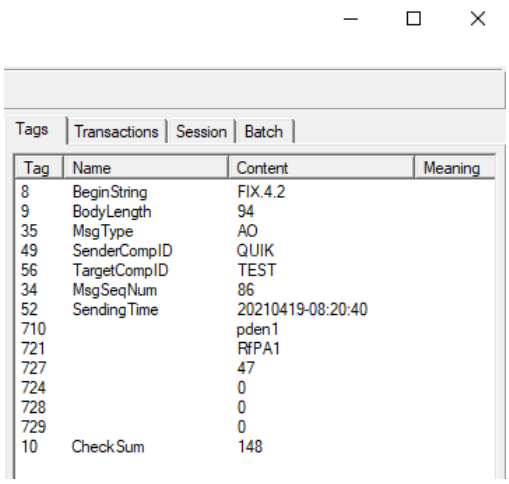


- **Request for Positions (35=AN)**. The request of client positions.

Name	Tag	Comment
MsgType	35	Message type. Value: AN
PosReqID	710	Client identifier of the request
PosReqType	724	Value: 0 – Positions
SubscriptionRequestType	263	Subscription type. Valid values: _ 0 – SNAPSHOT; _ 1 – SNAPSHOT_PLUS_UPDATES; _ 2 – DISABLE_PREVIOS_SNAPSHOT_PLUS_UPDATES
NoPartyIDs	453	Number of PartyID groups. If exists, then the value is 2
→ PartyID	448	Valid values depending on the PartyRole value: _ 3 – Client code for cash and securities limits; _ 13 – Firm code; _ 38 – Trading account for futures restrictions and positions
→ PartyIDSource	447	Value: D – Proprietary / Custom code
→ PartyRole	452	Valid values: _ 3 – ClientID; _ 13 – Order Origination Firm; _ 38 – Position account

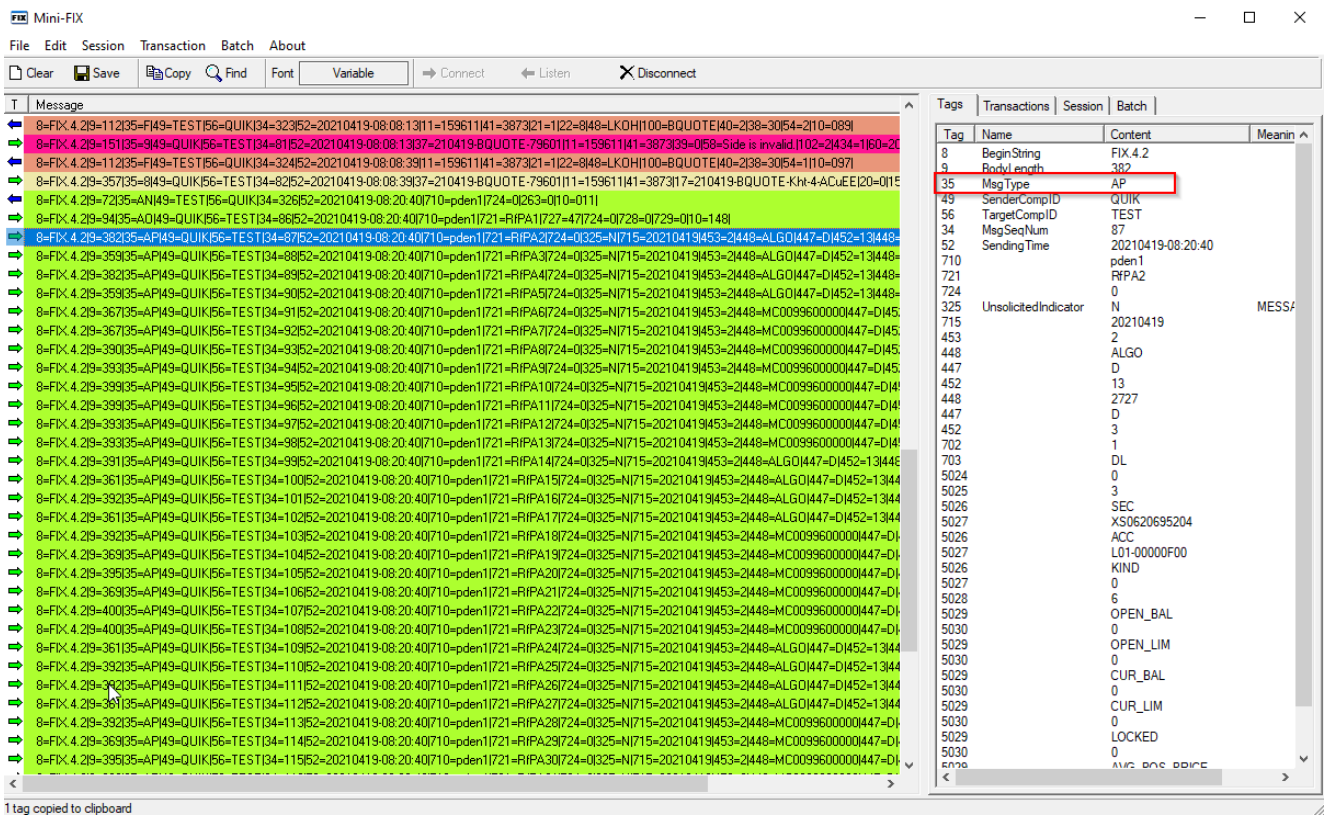


The **Request for Positions Ack (35=AO)** message is transmitted as a response.



Name	Tag	Comment
MsgType	35	Message type. Value: AO
PosMaintRptID	721	Client identifier of a response. Generated in format “RfPA<N>”, where N is an increasing integer
PosReqID	710	Client identifier of a request
PosReqType	724	Value: 0 – Positions
PosReqResult	728	Valid values:

Name	Tag	Comment
		_ 0 – Valid request; _ 1 – Invalid or unsupported request
PosReqStatus	729	Valid values: _ 0 – Completed; _ 2 – Rejected
Text	58	Text description of rejection reason
TotalNumPosReports	727	Total number of Position Report messages that will be received upon Request for Positions with the Snapshot (tag 263=0) subscription type



14.1 Most relevant examples of FIX messages

- **New Order Single (35=D)** – buy 30 lots of LKOH at limit price 5000 for the client 5253:

```
< 8=FIX.4.2|9=138|35=D|49=TEST|56=QUIK|34=359|52=20220114-
03:47:31|1=5253|11=2600|21=1|22=8|38=30|40=2|44=5000|48=LKOH|54=1|60=20220114-
03:47:31|100=BQUOTE|10=035|
```

- **New Order Single (35=D)** – sell 50 lots of SBER at market price for the client 5253:

```
< 8=FIX.4.2|9=130|35=D|49=TEST|56=QUIK|34=431|52=20220114-
05:47:45|1=5253|11=2638|21=1|22=8|38=50|40=1|48=SBER|54=2|60=20220114-
05:47:45|100=BQUOTE|10=192|
```

- **Order Cancel Request (35=F)** – order cancellation:

```
< 8=FIX.4.2|9=174|35=F|49=TEST|56=QUIK|34=479|52=20220114-
08:24:00|1=TST001|11=2663|15=RUB|22=8|38=50|41=2664|48=Tst|52=TIMESTAMP|54=1|60=20220
114-08:24:00|100=BQUOTE|40=2|37=220114-BQUOTE-49|10=124|
```

- **Market Data Request (35=V)** – request market data for the instrument LKOH:

```
< 8=FIX.4.2|9=118|35=V|49=TEST|56=QUIK|34=373|52=20220114-
04:05:52|22=8|48=LKOH|100=BQUOTE|262=2614|263=1|264=0|265=0|267=2|269=1|269=0|10=129|
```

- **Security Status Request (35=e)**.

```
< 8=FIX.4.2|9=78|35=e|49=TEST|56=QUIK|34=501|52=20220114
08:51:58|324=123456|263=1|22=8|48=Tst|10=251|
```

- **Security Definition Request (35=c)**.

```
< 8=FIX.4.2|9=88|35=c|49=TEST|56=QUIK|34=507|52=20220114
09:01:25|321=3|22=8|48=Tst|100=BQUOTE|320=12345|10=088|
```

- **Order Mass Status Request (35=AF)**.

```
< 8=FIX.4.2|9=67|35=AF|49=TEST|56=QUIK|34=494|52=20220114-
08:40:07|584=123456|585=7|10=116|
```

- **Order Status Request (35=H)**.

```
< 8=FIX.4.2|9=79|35=H|49=TEST|56=QUIK|34=498|52=20220114-
08:44:08|11=123456|37=220114-BQUOTE-50|10=076|
```

- **Request for Positions (35=AN)** – request positions of the client 5253:

```
< 8=FIX.4.2|9=128|35=AN|49=TEST|56=QUIK|34=413|52=20220114-
05:28:43|710=12345|263=0|448=5253|447=D|452=3|453=2|724=0|448=BrokerQuote|447=D|452=1
3|10=247|
```

- **Request for Positions (35=AN)** – request positions of clients:

```
< 8=FIX.4.2|9=71|35=AN|49=TEST|56=QUIK|34=414|52=20220114-
05:29:54|710=2345|724=0|263=0|10=002|
```

> **It is not recommended to request a position via FIX for QUIKFIX-BE (if FIX is used to implement an external trading application). In this case, we recommend using the Market data export module: arqatech.com/en/products/quik/modules/integration-solutions/market-data-export/**

15.FIX message configuration

> **FIX message configuration is supported only for FIX adapter and FIX Drop Copy program interfaces.**

An XML scheme is used for flexible configuration of the format of messages that are sent to and received by the Interface. Each FIX connection uses its own scheme.

Configuration is made in an *.xml file the path to which is specified in the **Schema** parameter.

```
[fix_connect_*]
…
Schema=Rules\Default30.xml
…
```

The scheme allows:

- prohibiting transmission of a tag specified in a protocol,
- copying a tag value to another tag (missing from a protocol),
- moving a tag value to another tag (missing from a protocol),
- setting the list of replacements for values of a tag transmitted and a tag received,
- setting a condition for rejection of an incoming message,
- transmitting non-standard tags (missing from a protocol) in response Execution Report messages;
- using regular expressions (according to the generally accepted ECMAScript syntax) in commands.

Commands are applied depending on the message direction:

- Y – a command is applied;
- N – a command is not applied.

Command	IN	OUT	Description
Email	N	Y	Add a tag with a value to the text of an email notifying of order entry or cancellation errors. For ExecutionReport, CancelReject messages only
Remove	N	Y	Prohibit transmission of a tag
Reject	Y	N	Reject an incoming message if the condition is true. For NewOrderSingle, OrderCancelReplaceRequest messages only
DiscardWholeMessage	N	Y	Do not send a message if the condition is true
Replace	Y	Y	Replace the tag value if the condition is true
CustomReplace	Y	Y	
ExternalReplace	Y	Y	Replace the tag value if the condition is true for another tag
MultiExternalReplace	Y	Y	Replace the tag value based on the set of conditions for two or more external tags
Clone	Y	Y	Copy the tag value to another tag
Swap	Y	Y	Exchange values of the current tag and the specified tag
Echo	Y	Y	Transmit non-standard tags (missing from the protocol) in Execution Report response messages. For NewOrderSingle, OrderCancelReplaceRequest messages only
Set	Y	Y	Set a constant value for a tag unconditionally
ForwardToTransField	Y	N	Set up transmission of the tag value in the transaction field addressed by an Info identifier. For NewOrderSingle, OrderCancelReplaceRequest and QuoteRequest messages only

Examples of some of the commands:

- **Remove** – exclude a tag from a message. For example:

```
<Rule MessType="8">
       <Tag ID="47">
              <Remove/>
       </Tag>
</Rule>
```

- **CustomReplace** – replace values of the tags that contain Cyrillic characters with a fixed value. For example:

```
<Rule MessType="8">
	<Tag ID="58">
		<CustomReplace If="ContainsCyrillic" Then="Transaction checking
	failed"/>
	</Tag>
</Rule>
```

- **Swap** – exchange values of the current tag and the specified tag. An example of swapping values of the tags [31] and [5007] if the value of the tag [1] is equal to RCB:

```
<Rule MessType="8">
	<Tag ID="31">
		<Swap CounterTag="5007">
			<Condition RefTagID="1" If="RCB"/>
		</Swap>
	</Tag>
</Rule>
```

- **Reject** – reject messages by specified condition. An example of rejecting a request for order entry if the value of the tag [64] SettlDate is not 20130401:

```
<Rule MessType="D">
	<Tag ID="64">
		<Reject If="20130401" CompareOperation="IsNotEqual" Message="Incorrect
value of settlement date"/>
	</Tag>
</Rule>
```

15.1 Error messages

When starting the interface, the scheme file format is validated. If a file is not found or if the file format is incorrect, the interface terminates with a diagnostic written to a log file:

- File is not found:

```
[ERROR] FIX-connection rules file does not exist <path='<path>'>
```

- File format is incorrect:

```
[ERROR] Couldn't load schema from file '<path>' [Reason='<reason, error
description>']
```

- The same target tag of the <Clone> command is set for more than one tag within a message:

```
[Warning] Clone target duplicate was detected for message '<message type>'. Clone command for tag #<tag number> has been left, duplicate for #<tag number> has been omitted.
```

16. Redundancy and monitoring of FIX interfaces

This section describes redundancy and monitoring of a connection between an adapter and a QUIK server.

16.1 Monitoring of connection between adapter and QUIK server

As FIX adapter / FIX Drop Copy connects to a QUIK server as an API user, such a connection can be monitored using the Qmonitor program.

If the section [Mail] is set up for FIX adapter / FIX Drop Copy, FIX adapter / FIX Drop Copy sends messages in case of disconnection from a QUIK server.

If there is a problem with an adapter itself (for example, with a platform where this adapter is installed), no email is sent.

An example of the section settings for sending emails:

```
[Mail]
EnableReporting=1
Server=198.125.010.020
Port=12350
FromAddress=abc@abc.ru
Address=yyy@yyy.ru
User=user1
Password=$55555
MsgTimespan=00:00:00-23:59:59
DisableRepeatingNotifications=1
ClientSchemeFileName=Desc\UserMessages.xml
```

Parameters description

- **EnableReporting** – indicates if the system administrator has to be notified of errors. Valid values:
  - 0 (default) – disabled;

- 1 – enabled.

- **Server** – SMTP server address.
- **Port** – SMTP server port.
- **FromAddress** – sender address.
- **Address** – recipient address. Value by default: "" (empty).
- **User** – user name if SMTP authorization is used. Value by default: "" (empty).
- **Password** – user password if SMTP authorization is used. Value by default: "" (empty).
- **MsgTimespan** – time period for sending notifications in format <HH:MM:SS-HH:MM:SS>. Value by default: 00:00:00-23:59:59.
- **DisableRepeatingNotifications** – prohibit resending of notifications in case of similar recurring events. Valid values:

  - 0 (default) – a notification is sent for every event;
  - 1 – a notification is not sent if the event status did not change since the last notification was sent.

- **ClientSchemeFileName** – path to a client XML scheme with texts of messages. Used to redefine values of attributes in a global XML scheme.

16.2 Hot redundancy of QUIK server connection

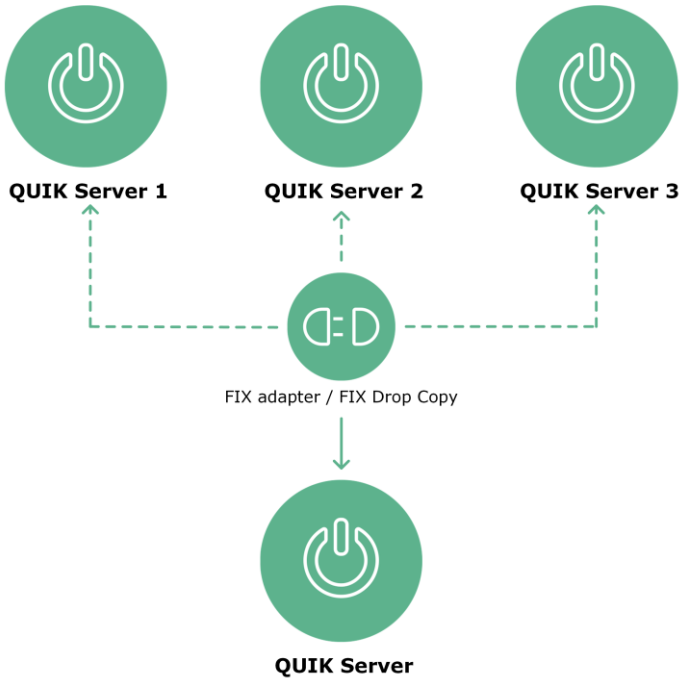
FIX adapter / FIX Drop Copy supports several sections of [QuikApi_*] view and one active connection set in the **ActiveQuikApiConnId** setting of section [General]:

- **ActiveQuikApiConnId** – identifier of a QuikApi connection used for reconnection of a program interface to another QUIK server that does not require restart of the interface.

Each time the interface connects or reconnects, it reads the value of the **ActiveQuikApiConnId** setting and initiates a connection using parameters of a specific section.

The **ActiveQuikApiConnSwitchTimeout** setting in section [General] defines the time period during which the interface tries to reconnect via the current active connection after a connection breaks down. When this time period ends, the current connection automatically switches to a backup connection addressed by the **SwitchToConnID** setting of the current connection.

An example of settings for cyclic switching to three connections:




```
[General]
ActiveQuikApiConnID=1
ActiveQuikApiConnSwitchTimeout=10

[QuikApi_1]
User=FIX
Password=qwe
Ini=QuikApi_1.ini
SwitchToConnID=2

[QuikApi_2]
User=FIX
Password=qwe
Ini=QuikApi_2.ini
SwitchToConnID=3

[QuikApi_3]
User=FIX
Password=qwe
Ini=QuikApi_3.ini
SwitchToConnID=1
```

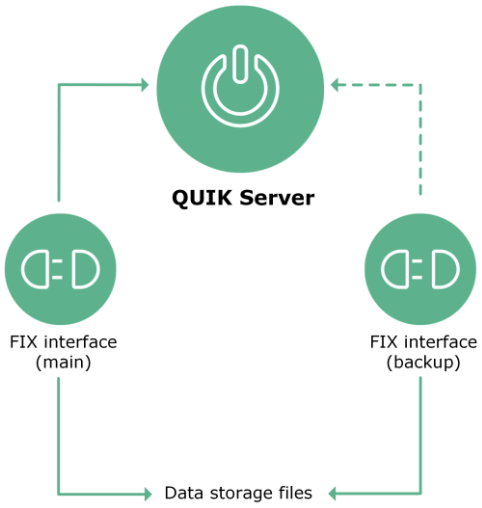
16.3 FIX adapter / FIX Drop Copy redundancy

For redundancy of adapters, copy the FIX interface directory to another platform. The interface data storage files must be stored on the network path.

The following files are data storage files:

- OrderHistory.data;
- orders.data;
- trades.data;
- storage.data;
- ustrades.data.

If there are problems with a main adapter (for example, platform crash), a backup adapter which is configured to work with storages of the main adapter located on the network is started.

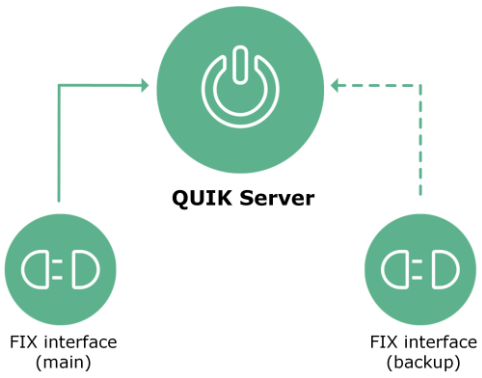


This scheme assumes that once a backup FIX interface is started it continues working from the moment when the problem occurred and uses the data storage files of the main FIX interface.

16.4 Order cancellation by number

FIX adapter allows cancelling and replacing orders that were submitted NOT using this interface instance if specific permissions on such orders are configured for a service user on the QUIK server. Any instance of the interface that have access to these orders can cancel or replace them on the QUIK server.

This option may be useful if there is a problem with the main adapter. Previously submitted orders can be cancelled using the backup (or any other) adapter.



To cancel an order, use the tag [37] with the date, class code, and order number specified.
The format of filling request identifiers to cancel or replace an order:

Name	Tag	Format
OrderID	37	<[Order registration date]-[Class code]-[Order number]>, where: _ [Order registration date] * is the date of order registration in the current session even if there was a date change on local time zone of the computer where the Interface is installed while registering an order. The maximum allowed length for an identifier is 40 characters

17.Maintenance schedule

> **NOTE: Daily restart is necessary for the FIX interface correct operation.**
> **The Interface stop and start must be performed considering the time of session change specified in the ClearingTime setting of the [General] section:**
> **the Interface must be stopped before and started after the session change time.**

For example:

```
[General]
…
ClearingTime=030000
…
```

18.Interface startup keys

18.1 -install

Install the interface service.

18.2 -remove

Delete the interface service.

18.3 -new

Start the interface service in the mode of deleting the storages of sent messages (client transactions history). Counters of incoming and outgoing messages are reset to 1.

19.Diagnostics of request rejection

Messages of order registration or cancellation can be rejected by the adapter, the QUIK server, or the trading system. If a message is rejected by the adapter, the text of message rejection is transmitted a FIX client.

19.1 New Order Single

Diagnostics	Description
ClOrdId is empty	Client's request identifier (tag [11]) is not set
ClOrdId is not unique	Client's request identifier (tag [11]) is duplicated within the session
Unsupported type of HandlInst. Valid values: 1, 3	Unsupported value is transmitted for the tag [21]
OMS class is not found	OMS class is not found
OMS class description is not found	The link to the file with OMS class description is incorrect or is not set
Unsupported algorithm	The link to the file with algo class description is incorrect or is not set
NEW_ORDER transaction is not defined for OMS class	The OMS class description has no order entry transaction
NEW_ORDER transaction is not defined for algorithm	The algo class description has no order entry transaction
Unsupported type of OrderRestrictions. Valid values: 5	Unsupported value is transmitted for the tag [529]
Unknown ExDestination [<EXDEST>]	The value of the tag [100] is neither a class code, nor a market code
Unknown RIC-code [<RIC>]	No correspondence is set between a transmitted RIC code and an ISIN code
ExpiryDate isn't supported for board <CLASS>	Transaction interface of the class <CLASS> does not allow transmitting the order expiration date
ExpiryTime isn't supported for board <CLASS>	Transaction interface of the class <CLASS> does not allow transmitting the order expiration time
Unsupported value of tag22 [IDSource]. Valid values:1, 2, 4, 6, 8, 100	Unsupported value is transmitted for the tag [22]
Specified security is not found [<SECID>]	Could not find an instrument
Ambiguously security specification	More than one instrument has a transmitted instrument identifier. Specify the code of a class or market

Diagnostics	Description
Security is locked now, try later	Instrument identifier on the QUIK server is currently unknown: no server connection
Trading by specified class is unavailable	Trading operations in this class are prohibited by the settings (the **classes** setting in section [quik])
Makler transaction for class <CLASS> is not available	Makler transaction for a class is not found
Unsupported type of OrdType. Valid values: 1, 2, 3, 4 and R	Unsupported value is transmitted for the tag [40]
ContraBroker is not defined	No counterparty is specified for negotiated orders
Stop-orders for class <CLASS> are not available	Stop orders are not available for this class
StopPx is not specified	Stop price is not specified
Unsupported type of TimeInForce. Valid values: 0, 1, 2, 3, 4, 6 and X	Unsupported value is transmitted for the tag [59]
Unsupported type of Side. Valid values: 1 and 2	Unsupported value is transmitted for the tag [54]
Nominal value of security is undefined	Instrument face value is not defined (for bonds only)
Invalid OrderQty(<QTY>): Nominal(<NOMINAL>) is not respected	Order volume transmitted is not multiple of face value (for bonds only)
Invalid OrderQty(<QTY>): LotSize(<LOTSIZE>) is not respected	Order quantity transmitted is not multiple of lot quantity
Iceberg-orders are not supported for specified security	Iceberg orders are not available in this class
Undescripted QuikApi-Error	A transaction is rejected by the QuikApi library but no description was received for that error
Value [<VALUE>] is unsupported for transaction field [<FIELD>]	A value transmitted for an enum field of a transaction is not supported
Rejected due to schema-level restriction	A request is rejected due to a restriction set in the scheme
Specified SettleDate is not allowed	Invalid settlement date is specified
Unsupported PegOffsetType	Unsupported value is transmitted for the tag [836]
Unsupported DiscretionOffsetType	Unsupported value is transmitted for the tag [842]
Invalid OrderQty(<ORDERQTY>): Must be integer	Unsupported value is transmitted for the tag [38]
SecurityType [<NNNN>] isn't supported. Valid values: FXSPOT, FXFWD, FXNDF.	Unsupported value is transmitted for the tag [167]
SecurityType [<FXSPOT>] isn't supported for security	A value unsupported by an instrument

Diagnostics	Description
[<FXClass_USDRUB>]. Supported value: FXNDF.	is transmitted for the tag [167]

19.2 Order Cancel Request

Diagnostics	Description
ClOrdId is empty	Client's request identifier (tag [11]) is not set
ClOrdId is not unique	Client's request identifier is duplicated within the session
Order is not found by OrigClOrdID: <ORIGCLORDID>	An order to be cancelled is not found by OrigClOrdID
SessionID mismatch, cancel is impossible	Session identifier assigned to an order when submitting it is different from the current one (OMS only)
OMS class description is not found	A link to the OMS class description file is invalid or not set
Unsupported algorithm	A link to the algo class description file is invalid or not set
Unsupported algorithm class	An algo class that corresponds to the algorithm is not found
Trading by specified class is unavailable	Trading operations in this class are prohibited by the settings (the **classes** setting in section [quik])
Order is already in pending cancel status	Cancellation of an order is in progress
Order is already in pending replace status	Replacement of an order is in progress
Order is already canceled	An order has already been cancelled
Order is already expired	An order is expired
Order is done for day	An order execution has already been confirmed
Order is in replaced status	An order has already been replaced
OrderQty is invalid	The quantity transmitted does not match the quantity in an order being cancelled
Nominal value of security is undefined	Instrument face value is not defined (for bonds only)
Invalid OrderQty(<QTY>): Nominal(<NOMINAL>) is not respected	Order volume transmitted is not multiple of face value (for bonds only)
Invalid OrderQty(<QTY>): LotSize(<LOTSIZE>) is not respected	Order quantity transmitted is not multiple of lot quantity
Side is invalid	Side transmitted does not match side of an order being cancelled

Diagnostics	Description
Unsupported type of Side. Valid values: 1 and 2	Unsupported value is transmitted for the tag [54]
OrderType is invalid	The type transmitted does not match the type of an order being cancelled
Undescripted QuikApi-Error	A transaction is rejected by the QuikApi library but no description was received for that error

19.3 Order Cancel Replace Request

Diagnostics	Description
ClOrdId is empty	Client's request identifier (tag [11]) is not set
ClOrdId is not unique	Client's request identifier is duplicated within the session
Order is not found by OrigClOrdID: <ORIGCLORDID>	An order to be replaced is not found by OrigClOrdID
Unsupported type of HandlInst. Valid values: 1, 3	Unsupported value is transmitted for the tag [21]
OMS class is not found	OMS class is not found
OMS class description is not found	A link to the OMS class description file is invalid or not set
Unsupported algorithm	A link to the algo class description file is invalid or not set
Unsupported algorithm class	An algo class that corresponds to the algorithm is not found
REPLACE_ORDER transaction is not defined for OMS class	The OMS class description has no order replacement transaction
CHANGE_ORDER transaction is not defined for algorithm	The algo class description has no order replacement transaction
Unknown ExDestination [<EXDEST>]	The value of the tag [100] is nor a class code, nor a market code
Unknown RIC-code [<RIC>]	No correspondence is set between a transmitted RIC code and an ISIN code
Unsupported value of tag22 [IDSource]. Valid values: 1, 2, 4, 6, 8, 100	Unsupported value is transmitted for the tag [22]
Specified security is not found [<SECID>]	Could not find an instrument
Ambiguously security specification	More than one instrument has a transmitted instrument identifier. Specify the code of a class or market

Diagnostics	Description
Security is locked now, try later	Instrument identifier on the QUIK server is currently unknown: no server connection
Trading by specified class is unavailable	Trading operations in this class are prohibited by the settings (the **classes** setting in section [quik])
Amendment of orders is not allowed	Order replacement is prohibited by the settings
Wrong HandlInst for OMS-order replace	Incorrect value is transmitted for the tag [21] (OMS only)
SessionID mismatch, replace is impossible	Session identifier assigned to an order when submitting it is different from the current one (OMS only)
Amendment of address orders is not allowed	Negotiated order replacement is not supported
Order is already in pending cancel status	Cancellation of an order is in progress
Order is already in pending replace status	Replacement of an order is in progress
Order is done for day	Order execution is already confirmed
Too late to cancel	An order has already been cancelled or replaced
Side is invalid	Side transmitted does not match side of an order being replaced
Unsupported type of Side. Valid values: 1 and 2	Unsupported value is transmitted for the tag [54]
Security is invalid	The instrument transmitted does not match the instrument of an order being replaced
Unsupported type of TimeInForce. Valid values: 0, 1, 2, 3, 4, 6 and X	Unsupported value is transmitted for the tag [59]
Unsupported type of OrdType. Valid values: 1, 2, 3, 4 and R	Unsupported value is transmitted for the tag [40]
OrdType mismatch	Attempted to replace a stop order with a common order or vice versa
Stop-order type mismatch	The stop order type transmitted does not match the type of a stop order being replaced
Nominal value of security is undefined	Instrument face value is not defined (for bonds only)
Invalid OrderQty(<QTY>): Nominal(<NOMINAL>) is not respected	Order volume transmitted is not multiple of face value (for bonds only)
Invalid OrderQty(<QTY>): LotSize(<LOTSIZE>) is not respected	Order quantity transmitted is not multiple of lot quantity

Diagnostics	Description
Iceberg-orders are not supported for specified security	Iceberg orders are not available in this class
Order cannot be reopened	Replacement of filled orders is prohibited by the settings
OrderQty is invalid	The quantity transmitted is less than filled quantity of an order being replaced
Unable to reopen filled order - new value of OrderQty must be greater than previous	The quantity transmitted is less than or equal to filled quantity of an order being replaced
ServiceField is too long	The combined length of values of the tags [100] and [207] exceeds length of a corresponding field in a transaction (OMS only)
Undescripted QuikApi-Error	A transaction is rejected by the QuikApi library but no description was received for that error
Rejected due to schema-level restriction	A request is rejected due to a restriction set in the scheme
Specified SettleDate is not allowed	Invalid settlement date is specified
Unsupported PegOffsetType	Unsupported value is transmitted for the tag [836]
Unsupported DiscretionOffsetType	Unsupported value is transmitted for the tag [842]

20. Equipment and communication channel requirements

- **Equipment**

The FIX Program Interface can be installed on the same computer with the QUIK server or on the other computer and connected to the server via local network or the Internet.

FIX program interfaces (FIX adapter, FIX Client Connector, FIX Drop Copy)	Processor Intel Xeon Gold 5118 or better, 2 GB RAM, 10 GB available hard disk space	OS Windows Server 2008/2012/2016/2019 (x64)

- **Communication channel**

To connect the FIX Program Interface to the QUIK server via the Internet it is sufficient to have a communication channel capacity of at least 9.6 Kbit/s.